

# PHASEZERO – Backend Java

## Practical Assignment

---

### 1. Objective

Build a small backend microservice in **Java** and **Spring Boot** that manages product catalogue data via **REST APIs**.

The goal of this assignment is to evaluate your skills in:

- Backend fundamentals (Java + Spring Boot)
  - REST API design and HTTP understanding
  - Data modelling and validation
  - Code structure and problem-solving approach
- 

### 2. Task Overview

You are required to create a Spring Boot application named: `phasezero-catalog-service`

Implement a **Product Catalogue** service that supports the following core operations:

1. Add a product
2. List all products
3. Search products
4. Filter products
5. Sort products
6. Calculate inventory value

You have the flexibility to decide the internal design (layers, patterns, classes), provided the solution is clean, maintainable, and understandable.

---

### 3. Data Model

Design a **Product** model that includes at least the following fields:

| Field Name        | Data Type | Description                      |
|-------------------|-----------|----------------------------------|
| <b>partNumber</b> | String    | Unique identifier (Business Key) |
| <b>partName</b>   | String    | Name of the product              |
| <b>category</b>   | String    | Product category                 |
| <b>price</b>      | double    | Unit price of the product        |
| <b>stock</b>      | int       | Current inventory count          |

Candidates may add additional fields (e.g., **id**, **brand**, **createdAt**) if deemed useful. Any added fields must be briefly explained in the final **README**.

### Business Rules

The following rules **must** be strictly enforced when creating or updating a product:

- **partName** must be stored in **lowercase** (normalized before saving).
- No duplicate **partNumber** is allowed.
- **price** and **stock** **cannot be negative**.

---

### 4. Required Functionalities & Endpoints

You must expose REST APIs to implement the following behaviours. Endpoint paths and HTTP methods should adhere to reasonable REST conventions (e.g., **/products**, **/products/search**).

| Operation                       | Description & Requirements                                                                                                                                                                                                            |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1) Add new product              | Create a product using the request body. Must apply all defined business rules and validations. On error (e.g., duplicate <code>partNumber</code> ), return an appropriate HTTP status and error message.                             |
| 2) List all products            | Return the full list of all products currently stored. The response must be in JSON format.                                                                                                                                           |
| 3) Search by name               | Search products whose <code>partName</code> contains a given text (case-insensitive). <i>Example: Searching for "filter" should match "hydraulic filter", "FILTER", etc.</i>                                                          |
| 4) Filter by category           | Return all products belonging to a specified category. (Method of passing category, e.g., path variable or query parameter, is at your discretion).                                                                                   |
| 5) Sort products by price       | Return the list of all products sorted by <code>price</code> in <b>ascending order</b> .                                                                                                                                              |
| 6) Return total inventory value | Compute the total inventory value using the formula: <b>sum of (<code>price</code> * <code>stock</code>)</b> for all products. This value must be returned from a dedicated endpoint (e.g., <code>/products/inventory/value</code> ). |

---

## 5. Validations (Mandatory)

Your APIs must return clear and consistent error responses when:

- `partNumber` already exists (Conflict).
- Required fields are missing or empty (Bad Request).
- `price` or `stock` are negative (Bad Request).

Choose appropriate HTTP status codes (e.g., 400 Bad Request, 409 Conflict) and design a consistent error representation (e.g., JSON object with `message`, `code`).

---

## 6. Technical Requirements

### Technology

- **Java 8 or 17**
- **Spring Boot**
- You may use any standard and common Spring Boot components (e.g., Spring Web, validation, testing utilities).

### Storage

Use **one** of the following options for data persistence:

- An in-memory data structure (e.g., `List`, `Map`) inside your Service layer.
- **OR** an in-memory database such as **H2**.

If an in-memory database is chosen, include a short note in the `README` explaining:

- Why it was chosen.
- How the schema is managed (e.g., JPA entities, `schema.sql`).

---

## 7. Code Structure Expectations

The solution must demonstrate proper separation of concerns. At minimum, the architecture should clearly separate:

- **Controller layer:** Handling HTTP requests and responses, mapping to service calls.
- **Service layer:** Implementing all business logic and rules.
- **Repository / Data Access layer:** Containing storage code (even if it's just an in-memory collection).

Use meaningful and descriptive class, method, and variable names throughout the codebase.

---

## 8. Deliverables

You must submit a **public GitHub repository link** containing:

1. **Source Code**
  2. A comprehensive **README** file that clearly explains:
    - How to build and run the application.
    - A short description of your design (layers, main classes).
    - The list of all available endpoints (path, HTTP method, brief description).
    - Example request/response for at least 2–3 key endpoints.
    - Any assumptions or limitations made during the development.
- 

## 9. Evaluation Criteria

Your assignment will be evaluated based on the following criteria:

| Area                        | Focus                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------|
| API Design & Correctness    | Endpoints, HTTP methods, responses, status codes.                                          |
| Business Logic & Validation | Proper handling and enforcement of defined rules and error cases.                          |
| Code Structure              | Separation of concerns, readability, maintainability, and naming conventions.              |
| Data Handling               | Effective use of collections or database, implementation of uniqueness, and normalization. |
| Documentation (README)      | Clarity of explanation, ease of building, running, and testing the application.            |

---

## 10. Notes

- The focus is on clarity of thinking and clean structure, which are more important than complexity or excessive features.
- Aim for a solution that you can confidently explain and extend in a follow-up discussion.